自我介绍

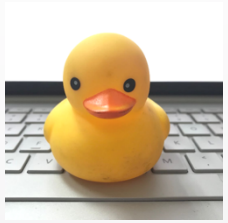
- 2014年NOI金牌
- 2018年CCSP第四名
  （CCF大学生计算机系统与程序设计竞赛）
- 为信息学竞赛做过一些贡献
  （在 NOIP/WC/APIO/NOI/清华集训
  中均有过命题或授课）
- 现为清华大学计算机系本科生，
  预计下学期开始攻读清华计算机系博士。



- 可通过以下方式联系我：



QQ/微信



评测鸭 duck.ac



UOJ 用户群

评测系统剖析与选手参赛策略

清华大学 计算机科学与技术系 ~~评测鸭创始人~~ 王逸松

wangyisong1996@gmail.com

Q & A & Q

- **Q：这节课讲什么？**
- A：评测系统的原理、选手的参赛策略。
- **Q：听说第一课堂的课很难？**
- A：是的。不过更难的在后面，听完这节课再去第二课堂还来得及。
- **Q：我想写一个评测系统！**
- A：请认真听课！然后自学成才！
- **Q：明天要考这个吗？**
- A：醒醒，明天没考试。
- **Q：Zzz…**
- A：（祝您冬眠愉快）

回顾：信息学竞赛的题目

- 给定一个问题，要求编写程序，对特定的输入能产生特定的输出……
- 题目与成绩直接挂钩——通过评测。

【题目描述】

11328 年，C 国的科学家们研发了一种高速传送通道，可以在很短的时间内把居民从通道的一端送往另一端，这些通道都是双向的。

美中不足的是，这种传送通道需要进行大量的维护和检修。经过规划，C 国总统决

【输入格式】

从文件 ***tunnel.in*** 中读入数据。

输入文件的第 1 行包含一个正整数 n，表示传送站的个数，传送站从 1 到 n 编号；

输入文件的第 2 到第 n 行，每行包含 3 个数 u, v, w，表示第一组通道中有一条连

【输出格式】

输出到文件 ***tunnel.out*** 中。

输出文件共 1 行，包含一个整数，表示最大的使用者数量之和。

图片来源：CCF NOI 2018 冬令营 正式赛

回顾：信息学竞赛的流程



选手

准备比赛 → 参加考试 → 焦急地等待 → 查看成绩 → 领奖



出题人

准备题目 → 监考 → 焦急地等待 → 讲题



评测人员

准备评测系统 → 监考 → 评测 → 受理申诉

评测系统

图片来源：https://upload.wikimedia.org/wikipedia/commons/a/af/Tux.png

评测是信息学竞赛的重要环节



选手

准备比赛 → 参加考试 → 焦急地等待 → 查看成绩 → 领奖



出题人

准备题目 → 监考 → 焦急地等待 → 讲题



评测人员

准备评测系统 → 监考 → 评测 → 受理申诉

评测系统

图片来源：https://upload.wikimedia.org/wikipedia/commons/a/af/Tux.png

评测系统

- 输入：考试的题目、选手的程序
- 输出：每个选手每道题的成绩

- “主文件夹的对应题目目录下”
- /home/evaluate/ansbackup
- /home/evaluate/result

- **评测系统对评测起到了关键作用！**

评测系统对整个竞赛起到了关键作用



选手

认真准备考试 → 评测系统挂了 → 无法取得理想的成绩 → 没学上



出题人

出神题 → 评测系统不支持 → 出坑题



主办方

精心准备比赛 → 评测系统挂了 → 大量选手不满

图片来源： https://www.ccf.org.cn/images/logo.png

本节课的内容

- **评测系统的分析与实现**
  - 什么是评测系统
  - 测什么
  - 怎么测
  - 有什么特性
- **现有评测系统与选手参赛策略**
  - 离线评测系统
  - 在线评测系统
  - 参赛策略讨论

评测系统的分析与实现

什么是评测

- **最基本的评测：**
  - 给定一个源程序
  - 给定一个输入文件
  - 给定一个答案文件
  - 给定一个时间限制
  - 给定一个内存限制

  - 要求编译、运行源程序，并将其输出与答案文件比对。
  - 此外，要求被测程序运行时不能超出时间限制和内存限制。

什么是评测系统

- **自动化（或半自动化）地完成下列功能：**
  - 给定一个源程序
  - 给定一个输入文件
  - 给定一个答案文件
  - 给定一个时间限制
  - 给定一个内存限制

  - 要求编译、运行源程序，并将其输出与答案文件比对。
  - 此外，要求被测程序运行时不能超出时间限制和内存限制。

测什么

- **输入**：选手源程序，输入文件，答案文件，时间/内存限制。
- **输出**：编译、运行结果、正确性。

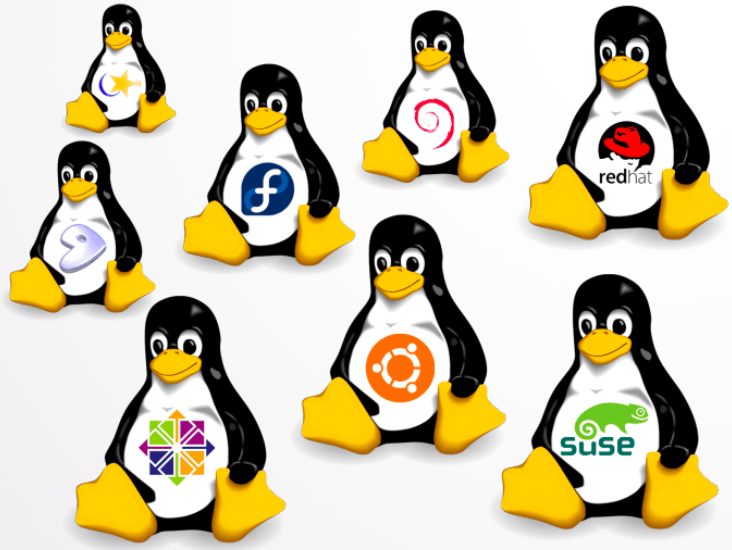
- **编译**：使用评测系统中的编译器，是否能将选手源程序编译成功。
- **运行时间**：从选手程序启动到结束，这一段过程中选手使用的时间。
- **内存用量**：在选手程序的整个运行周期，最多同时使用了多少内存。
- **正确性**：选手程序产生的输出文件是否与答案文件一致。

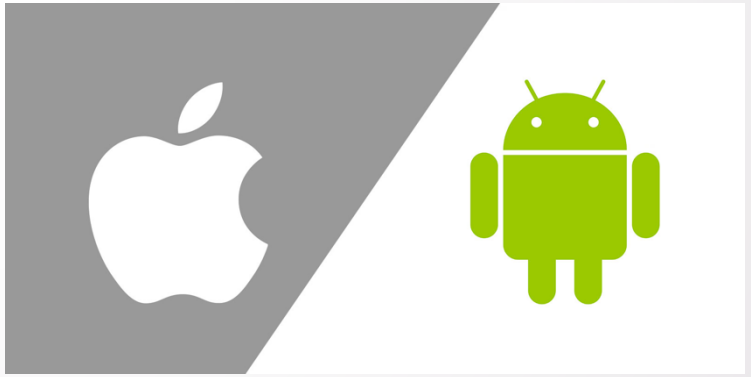
- `// C++, 1s, 512MB, 误差不超过 1e-6 算作正确`

怎么测

- 首先确定评测系统运行在何种操作系统上。



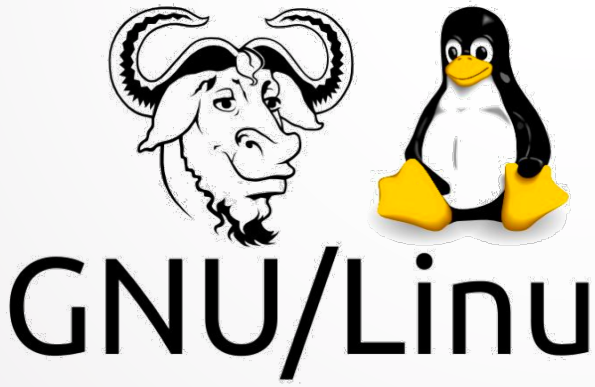


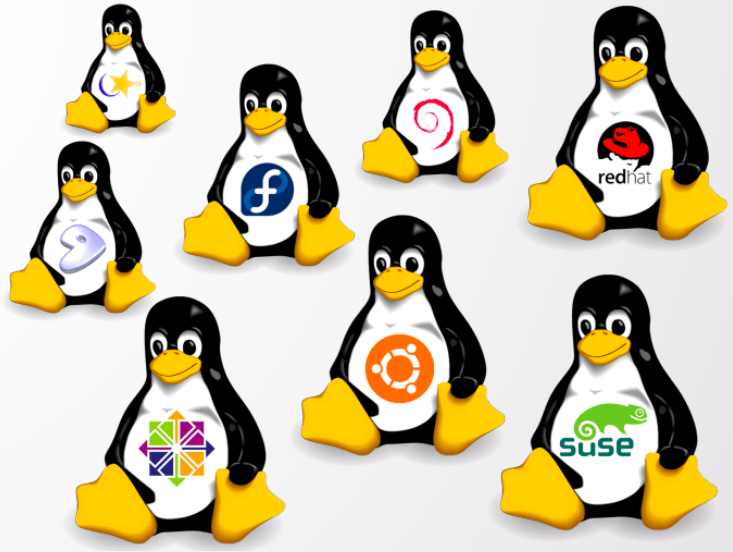


https://www.supinfo.com/articles/single/4881-evolution-of-microsoft-operating-systems
https://www.fiverr.com/arunpyasi/create-a-custom-linux-distro-for-you
https://buildfire.com/ios-android-users/

GNU/Linux

- **GNU**：一个自由的操作系统，其内容软件完全以GPL方式发布。
- GNU's Not Unix!
- **Linux**：一种自由和开放源代码的类UNIX操作系统。
- Linux 有很多发行版(distros)。我们通常使用运行在 x86-64 上的 Ubuntu。





https://zh.wikipedia.org/wiki/GNU
https://zh.wikipedia.org/wiki/Linux
http://www.linuxandubuntu.com/home/what-is-gnu-in-gnulinux

如何使用 Linux

- 首先打开“终端”，然后可以通过命令行，与系统交互。
- 常用命令：
  - `uname -a`：显示当前计算机名称、内核版本号等信息。
  - `echo`：显示一个字符串。
  - `./prog`：运行 prog 这个程序。
  - `ls`：显示当前目录下的文件列表。
  - `pwd`：显示当前目录的路径。

```
pigeon@pigeon-desktop:~$ uname -a
Linux pigeon-desktop 4.15.0-43-generic #46~16.04.1-Ubuntu SMP Fri Dec 7 13:31:08 UTC 2018
 x86_64 x86_64 x86_64 GNU/Linux
pigeon@pigeon-desktop:~$ echo "233"
233
pigeon@pigeon-desktop:~$ ./hello
hello world
```

怎么测

- **输入**：选手源程序，输入文件，答案文件，时间/内存限制。
- **输出**：编译、运行结果、正确性。

- **编译**：如何使用评测系统中的编译器？
- **运行时间**：如何运行选手程序，并测量时间？
- **内存用量**：如何运行选手程序，并测量内存用量？
- **正确性**：如何比较输出文件与答案文件？

编译

- 使用 <编译器> <选手源程序> <编译选项> 进行编译。
- **编译器：** gcc / g++ / fpc
- **选手源程序：** main.c / main.cpp / main.pas
- **编译选项：** -O2
- 编译结束后会给出错误信息和警告信息！

开启 -O2 优化 可执行文件存为 main

```
pigeon@pigeon-desktop:~$ g++ main.cpp -O2 -o main -Wall
main.cpp: In function 'int main()':
main.cpp:3:12: error: 'puts' was not declared in this scope
  puts("233");
           ^
main.cpp:2:6: warning: unused variable 'a' [-Wunused-variable]
  int a = 233;
      ^
```

显示所有警告

运行时间

- 使用 ./prog 来运行一个程序。
- 使用 time ./prog 来测量一个程序的运行时间。

- real 还是 user 还是 sys ？

```
pigeon@pigeon-desktop:~/test$ time ./a.out

real    0m0.724s
user    0m0.196s
sys     0m0.528s
```

运行时间

- 使用 `time ./prog` 来测量一个程序的运行时间。

- real: 真实时间
- user: 用户态时间 —— 选手代码的运行时间
- sys: 内核态时间 —— 操作系统服务（文件读写？网络访问？）的运行时间

```
pigeon@pigeon-desktop:~/test$ time ./a.out

real    0m0.724s
user    0m0.196s
sys     0m0.528s
```



内存用量

- 通过 /proc/<pid>/status 可查看一个进程的各项状态，其中包括了内存用量。
- <pid>：进程号

- 什么是进程号？什么是进程？？进程和程序是什么关系？？？

进程与程序

- **程序（可执行文件）：**存在文件系统中的一段代码（由CPU指令构成）。
- **进程：**在操作系统中运行的程序。

- 一个程序可以被多次运行，因此一个程序可以对应多个进程！



思考熊.gif

GIF 图像－4 KB
创建时间 2019/1/22
修改时间 2019/1/22

程序



进程

内存用量

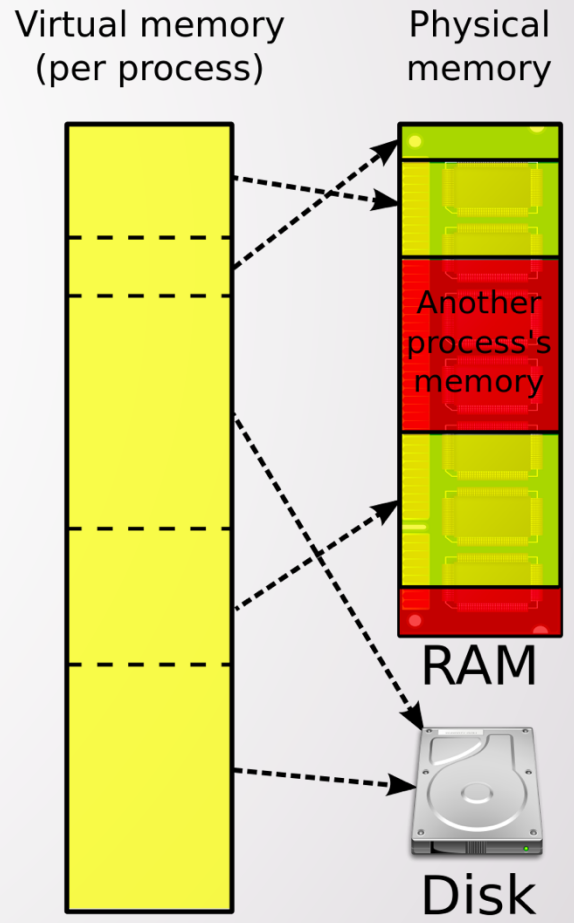
- 通过 /proc/<pid>/status 可查看一个进程的各项状态，其中包括内存用量。
- <pid>：进程号，可在启动进程时、进程运行时获得。
- pidof <program>：返回 <program> 的所有进程的进程号。
- 也可用 getrusage 函数来获得以上信息！（请通过 man getrusage 来学习）
- 这么多内存信息，测哪个呢？

```
VmPeak:     4352 kB
VmSize:     4352 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:       720 kB
VmRSS:       720 kB
RssAnon:              68 kB
RssFile:             652 kB
RssShmem:              0 kB
VmData:      180 kB
VmStk:       132 kB
VmExe:         4 kB
VmLib:      1952 kB
VmPTE:        56 kB
VmSwap:        0 kB
```

物理内存与虚拟内存

- 操作系统可以同时运行多个进程，每个进程都有独立的地址空间。
- 但是物理内存只有一份！
- 为了实现这个特性，引入“虚拟内存”。
  - 以 Page（4KB）为单位，建立分块映射。

- 虚拟内存到物理内存的映射
- 虚拟内存到磁盘文件的映射（swap, pagefile.sys）
  - 同时运行更多的进程！
- 不同进程的虚拟内存指向同一块物理内存
  （共享内存, 动态链接库）



https://en.wikipedia.org/wiki/File:Virtual_memory.svg

内存用量

- 运行 `man proc`，查看关于 `/proc/<pid>` 的帮助信息。

- *VmPeak*: Peak virtual memory size.（峰值虚拟内存大小）
- 这就表示被测程序“用了”多少内存！

```
VmPeak:     4352 kB
VmSize:     4352 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:       720 kB
VmRSS:       720 kB
RssAnon:              68 kB
RssFile:             652 kB
RssShmem:              0 kB
VmData:      180 kB
VmStk:       132 kB
VmExe:         4 kB
VmLib:      1952 kB
VmPTE:        56 kB
VmSwap:        0 kB
```

正确性

- 如果你的程序只有几个数的输出，那么人工比较是个好办法。
- 如果你的程序输入和输出10万个数？？？

- 输入输出重定向：
  - `./a.out < in > out`
- 比较两个文件：
  - `diff <a> <b>`
  - `-s`：当一样时，输出“Files <a> and <b> are identical”
  - `-b`：将任意多个连续空格视为一个

```
pigeon@pigeon-desktop:~$ diff out ans -s -b
Files out and ans are identical
```

时间和内存限制

- 我们现在虽然能够测量程序的时间和内存，但是没有对其进行限制。

- 如果一个程序死循环了？
- `ulimit -t` 时间限制（秒数）

- 如果一个程序占用过多内存了？
- `ulimit -v` 内存限制（KB数）

```
pigeon@pigeon-desktop:~$ ulimit -t 3
pigeon@pigeon-desktop:~$ ./main
Killed
pigeon@pigeon-desktop:~$ time ./main
Killed

real    0m3.000s
user    0m3.000s
sys     0m0.000s
```

- 注意：限制的是什么时间、什么内存，请自行研究。

总结

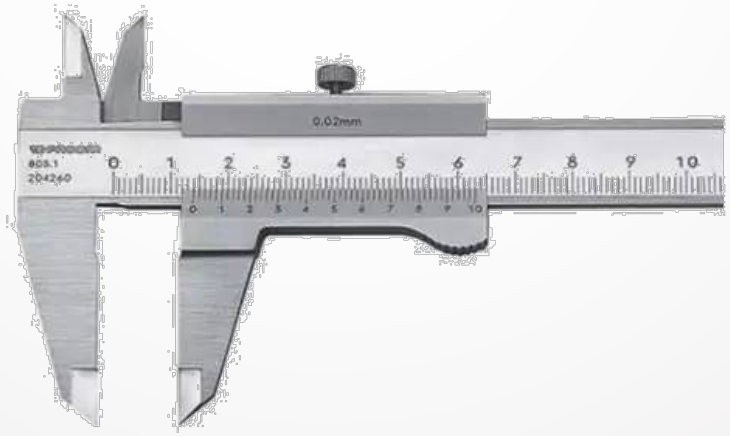
- 我们实现了一个最简单的评测系统！

- 输入：程序、输入文件、答案文件、时间内存限制
- 编译：gcc/g++/fpc
- 时间测量：time
- 内存测量：/proc/<pid>/status
- 正确性：输入输出重定向、diff
- 时间内存限制：ulimit

评测系统的特性

精确性

- 精确性是指测量的精度。分度值越小，对应的精度越高。
- 使用 `time` 可测得 ms 级别精度的时间。
- 使用 `/proc/<pid>/status` 或 `getrusage` 可测得以 4KB 为单位的内存。
- 1ms 和 4KB，对于大多数【评测】的场景已经足够。
- 若要实现更高的精度，其实没有意义。（为什么呢？）



图片来源：http://www.sohu.com/a/207108925_825115

稳定性

- 稳定性是指多次测量的结果的稳定：时间、内存、输出结果。
- 多次运行被测程序，内存用量和结果相对稳定，但可能发现时间变化了很多！

平均时间(s)	极差(s)	极差	标准差
1.299	0.123	9.47%	3.32%

Test #1:	score: 0	Wrong Answer	time: 1362ms	memory: 391032kb
Test #2:	score: 0	Wrong Answer	time: 1328ms	memory: 391032kb
Test #3:	score: 0	Wrong Answer	time: 1315ms	memory: 391028kb
Test #4:	score: 0	Wrong Answer	time: 1245ms	memory: 391028kb
Test #5:	score: 0	Wrong Answer	time: 1327ms	memory: 391028kb
Test #6:	score: 0	Wrong Answer	time: 1273ms	memory: 391028kb

http://uoj.ac/submission/309674, “提交记录 #309674 - Universal Online Judge”（背景颜色已修改），截图于 2018-12-26.

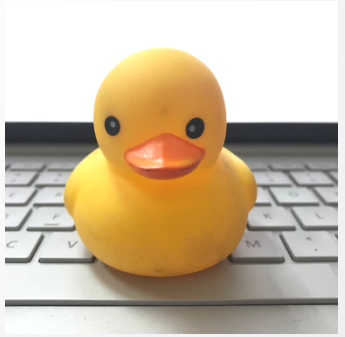
为什么时间不稳定

- （1） 程序运行时需要进行“系统调用”，虽然系统部分不计入时间，但每次切换会带来一个不确定开销。
- （2） CPU和内存之间有一些“缓存”，它们在程序每次运行时，状态并不确定，因此对程序性能的影响也不确定。
- （3） CPU是多核的，能同时运行多个进程。正在评测时，你可能正在上网看视频，这时其他进程会不确定地影响内存和缓存的正常使用。

如何提高稳定性

- 一种可能的想法：
- （1） 去掉系统调用！
- （2） 让缓存的行为变得更确定！
- （3） 不要运行其他程序！

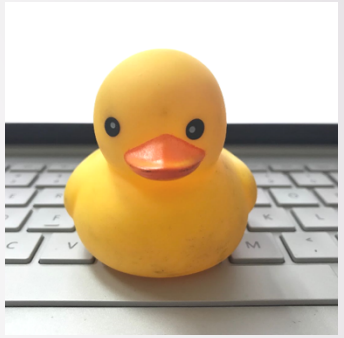
- 评测鸭（Judge Duck Online）—— 一个实现了上述想法的评测系统。
- 欢迎体验！网址：duck.ac



图片来源：https://duck.ac

评测鸭（Judge Duck Online）

- 一个稳定、精确的评测系统。
- 请访问 https://duck.ac，以了解更多技术。



- 只要您在 duck.ac 上注册并 AC 【测测你的 A+B】，即可获得小黄鸭一只！
- 帮助您实现算法和 debug，以及缓解压力~ （可带入考场）
- 注意：数量有限，先到先得



安全性

- 防止选手攻击评测系统，影响其正常运行：
  - 防止选手把评测机关了
  - 防止选手窃取答案文件
  - 防止选手修改自己的得分
  - ……



- 实现：
  - ptrace：监视被测进程的每个系统调用，阻止可疑的行为。
  - 虚拟化：在虚拟机或者类似环境中运行被测程序，实现资源的完全隔离。

图片来源：https://diamondbusiness.net/2017/04/28/new-nature-of-security/

高效性

- 评测系统通常为比赛服务，或作为在线评测网站的后端，因此需要高效。
- 实现：使用多台评测机。
- 某OJ：要比赛了，购买30台云服务器作为评测机，比赛结束了就关掉。





易用性

- 良好的图形界面
- 良好的用户服务
- ……
- 以评测为核心，提供多样化的服务
- 社区化的在线评测系统几乎已成标配



从以下网站截图，于2019-01-24：
https://www.lydsy.com/JudgeOnline/, https://codeforces.com/, https://loj.ac, http://uoj.ac,

总结

- 什么是评测
- 什么是评测系统
- 如何实现评测系统
- 评测系统的特性：精确性、稳定性、安全性、高效性、易用性



Have a break

- 课件将于结束后在 https://wys.life/201901-Judge-Lecture.html 发布。
- 你也可以扫描图中的二维码 ↓

- 小黄鸭领取指南：
  - 请排队领取，并提供你的【提交记录编号】。
  - 数量有限，先到先得。



现有评测系统与选手参赛策略

- **现有评测系统**
  - 离线评测系统
  - 在线评测系统
  - 评测系统的更多应用
- **选手参赛策略**
  - 评测系统与参赛策略
  - 不同赛制下的参赛策略

现有评测系统

离线评测系统

- 不依赖网络即可运行，可在本地进行测试。
- 也有一些支持局域网内联机。
- 多用于选手的训练，和一些考后统一评测的比赛。

- 优点：配置、使用方便
- 缺点：并非全自动，只适用于小规模测试



Cena



Lemon



Arbiter

图片来源：https://oi-wiki.org/intro/judgers/

在线评测系统

- 依赖网络的评测系统，通常自动化程度更高。
- 多数以网站的形式给选手提供服务。
- 当选手通过网页提交一份程序时，后台的评测机将自动完成评测任务，并在网页给予反馈。
- 适用于网络比赛，选手可以在任何地方参赛。









网络比赛？

- 网络比赛的选手并非需要来自互联网，也可以是同一地理位置的！
- 同一个学校的训练（断网），甚至……清华集训也使用在线评测系统！



训练平台？

- 在线评测系统也可提供【题库】等服务，甚至成为选手的训练平台。
- 注意：某些训练平台甚至是收费的！

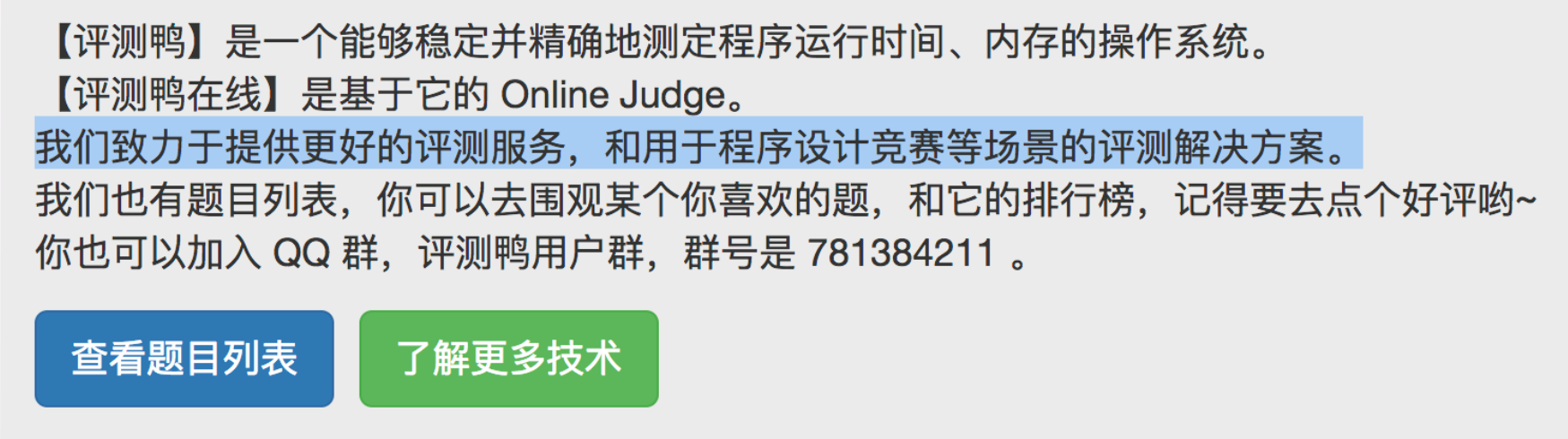


https://class.luogu.com.cn/course，截图于 2019-01-24。

一个脑洞：Judge as a Service

- 很多人、很多学校都有自己的在线评测系统。
- “评测”是在线评测系统的核心，但是大家都实现一样的评测有必要吗？
- 更重要的，难道不是评测系统的其他功能？

- 脑洞：提供“评测”服务，给其他评测系统使用！



https://duck.ac，截图于 2019-01-24

另一个脑洞：OJ as a Service

- 不想实现自己的评测系统，但是想管理自己的题目资源？
- 直接用现成的评测系统？不会安装？不想管服务器？

- 脑洞：提供一套完整的在线评测系统解决方案，用户只需上传资源！



https://vijos.org/wiki/help#domain ，截图于 2019-01-24

总结

- 离线评测系统
- 在线评测系统
- 在线评测系统的多种用法（Judge as a Service, OJ as a Service, …）



选手参赛策略

评测系统与参赛策略

- 技术限制了比赛的形式！

- 离线评测系统：
  - 比赛开始时选手可以得到题目和样例文件
  - 比赛过程中选手几乎只能得到答疑
  - 比赛结束后统一进行评测
- 在线评测系统：
  - 比赛过程中选手可以随时提交，随时得到反馈！

- **本质上，评测系统决定了赛制，从而对应了不同参赛策略！**

各种各样的赛制

- NOI —— 离线，可能提供大样例
- IOI —— 在线，记录最后一次提交，可查看得分，不可查看排行榜
- ICPC —— 在线，统计AC题数，和按提交次数/时间计算的“罚时”，有榜
- Codeforces —— 按提交次数/时间计算每题得分，有榜，可随时“Hack”
- TopCoder —— 同 Codeforces，但有专门的时段来 Hack
- UOJ —— 同 IOI，但是不可查看得分，以及可查看“测样例”的排行榜
- ……

甚至有人对赛制建了模

D. Google Code Jam

time limit per test: 2 seconds
memory limit per test: 256 megabytes
input: standard input
output: standard output

Output

Print two real numbers — the maximum expectation of the total points and the corresponding minimum possible time penalty expectation. The answer will be considered correct if the absolute or relative error doesn't exceed 10^{-9}.

- 输出两个实数 — 最大期望总得分，以及对应的期望最小罚时。

https://codeforces.com/contest/277/problem/D ，截图于 2019-01-24

如何构建自己的参赛策略

- 弃疗法：不管什么比赛，输出样例就走人。
- 暴力法：每道题写个暴力！（不会写暴力怎么办？？？）
- 顺序法：for 每道题 { 写暴力; 写正解; }

- 请大家发表高见或吐槽！！



对于离线为主的赛制

- 这类赛制不支持中途提交和查看结果。
- 但作为一种人情味的关怀，某些题目提供了“大样例”。
- 作为选手，需要尽可能使自己的程序不出问题。

- 一些可能的方案：
  - “开场每个题都写暴力”
  - “写完正解后要‘对拍’”
  - “用暴力跑小数据，用不确定能否通过的算法跑大数据”
  - “大样例不可信，需要自己造”（可是造数据又要花时间？）



对于在线的类OI赛制

- 这类赛制支持随时提交，并查看结果，但不一定有排行榜。
- 每道题目可能有若干“子任务”，对应不同的得分和难度。
- 作为选手，需要规划比赛时间，获得尽可能高的分数。



- 一些可能的方案：
  - ~~“开场每个题都写暴力”~~（什么时候需要写暴力？）
  - ~~“写完正解后要‘对拍’”~~（什么时候需要对拍？）
  - “用暴力跑小数据，用不确定能否通过的算法跑大数据”
  - “按照 期望得分/期望用时 选择题目做”

对于在线的类ICPC赛制

- 这类赛制支持随时提交，并查看结果，且有排行榜。
- 只统计AC题数，和基于提交次数和时间的“罚时”。
- 作为选手，需要规划比赛时间和解题顺序，并考虑组队策略。



- 一些可能的方案：
  - “跟着榜做题” （但是会落后于榜？）
  - “先找简单题/难题做，拿下多个一血”
  - ~~“女装”~~
  - “和队友们同时攻克一道题 vs. 每人开一道题”

其他想法？

- **有些东西不能让你得分，但是能减少某些题的期望时间：**
  - “学习一下 Python 怎么用”
- **有些东西不能让你得分，但是能玄学影响某些题的期望时间：**
  - “上个厕所，想到了一个算法”
  - “上个厕所，路上发现自己的算法是假的”
  - “上个厕所，想到了一个假算法，但是不知道是假的”
  - “上个厕所，什么都没发生，但是缓存空了” （即，忘记了之前的思考内容）
- **善于提问，以及善于利用提问系统！**
  - 说不定能问到评测相关的信息！
  - 说不定能获得加时！（如：题目出锅了）

特殊情况？

- 样例里本应是 \n，出现了 \r\n ？？？
- IOI赛制，本应有“子任务”，提交上去发现按测试点给分？？？
- 评测机配置与本机很不一样？？？
- IOI赛制，大家一起交暴力，评测系统挂了？？？
- 出题人发布公告，结果一半多的人没听见？？？



永恒的议题：策略 / 赛制 / 比赛

- 什么样的赛制应该使用什么样的策略？
- 什么样的比赛（或考试）应该使用什么样的赛制？
- 比赛（或考试）的目的是什么？想要区分什么？想要选拔什么？
- ……

完结撒花！

- 评测系统与赛制密切相关，赛制决定了参赛策略。
  - OI赛制、IOI赛制、ICPC赛制……
- 比赛（或考试）目的决定了赛制。

- 作为一个 Oier，既要熟悉评测系统的知识，也要了解 OI 的发展动向，才能掌握自己的命运啊！



Thanks !

- 感谢CCF给我此次交流机会。
- 感谢大家的认真聆听。
- 感谢各评测系统的开发者、维护者们，你们为 OI 提供了重要的技术支持！

- 祝大家考试顺利！~~冬眠愉快！~~

